

# HARVICS OS — SYSTEM MASTER CURSOR SPECIFICATION

## Complete Build Authority · Modules 01–09 + CRM Amendment

**Classification:** Confidential — Internal Build Document

**System:** Harvics OS / Harvics Universe

**Authority:** This file governs all Cursor actions on Modules 01–09.

**Benchmark:** 05-CRM-SALES module sets the depth standard for all other modules.

**Rule:** Read this file completely before touching any module. Read the relevant module section before touching that module's code.

---

## MASTER READ ORDER FOR CURSOR

None

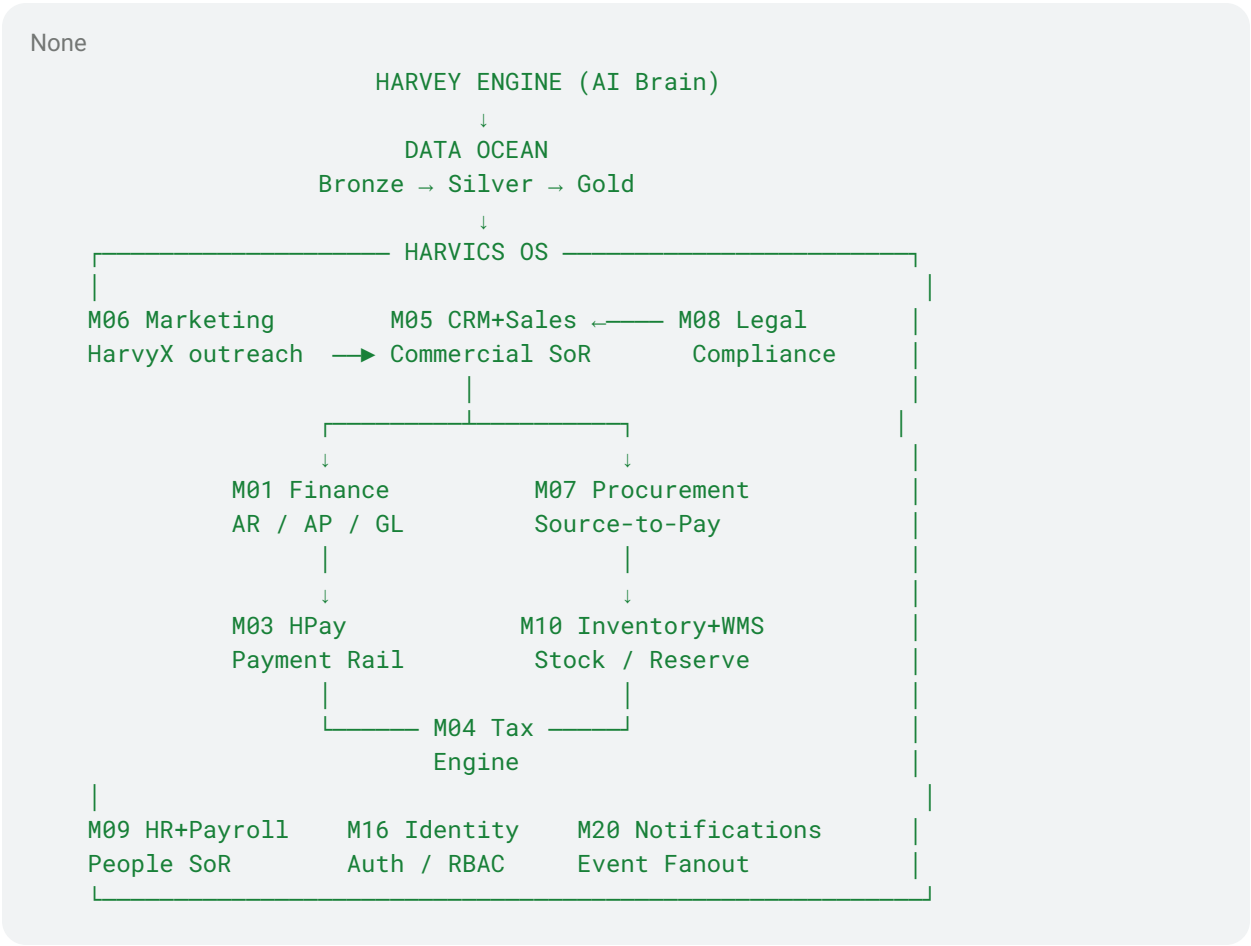
1. This file (system-wide rules + your target module section)
2. MASTER\_CURSOR\_CONSTITUTION.md (repository root)
3. module-registry.json (canonical status flags)
4. Existing Prisma schema (what is actually modelled)
5. Existing repository code (what is actually implemented)

**Specification ≠ Implementation. Implementation ≠ Production.**

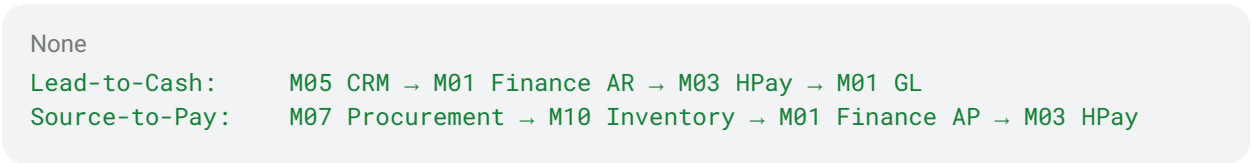
Status labels used throughout:

- **IMPLEMENTED** — verified in Prisma schema and running API
  - **SPECIFIED** — architecturally defined, not yet built
  - **SCAFFOLD** — generic factory only, not domain product
  - **TARGET** — future intended capability
  - **UNKNOWN** — spec exists; implementation unverified
  - **TBD** — source is silent; decision required before building
-

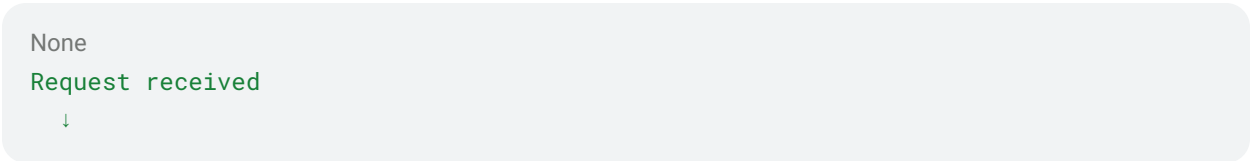
# SYSTEM ORGANISM MAP



## Two primary commercial highways:



## GOLDEN MUTATION SPINE (applies to every module)



- |                         |                                                                                |
|-------------------------|--------------------------------------------------------------------------------|
| 1. Authentication       | - valid Bearer JWT, not expired                                                |
| 2. Authorisation        | - actor has required permission (RBAC)                                         |
| 3. Tenant isolation     | - tenantId from JWT, NEVER from request body                                   |
| 4. Schema validation    | - Zod schema, typed inputs, fail fast 400                                      |
| 5. Master data reads    | - load required entities, check existence                                      |
| 6. Business rules       | - domain invariants, status machines                                           |
| 7. AI decision          | - if applicable, typed tool → result (advisory unless spec says authoritative) |
| 8. Neural Governance    | - Legal / Budget / Contract / Security / Compliance                            |
| 9. Human approval       | - if governance requires or threshold exceeded                                 |
| 10. Transactional write | - single DB transaction, idempotency key where required                        |
| 11. Domain event        | - publish to event bus                                                         |
| 12. Notification        | - relevant actors notified via M20                                             |
| 13. Audit record        | - immutable, includes governance + AI ref                                      |
| 14. Response            | - typed envelope                                                               |

---

## STANDARD API ENVELOPE

TypeScript

```
// Single
{ data: T, meta: { requestId: string, timestamp: string } }

// List
{ data: T[], meta: { requestId: string, timestamp: string, page: number, perPage:
number, total: number } }

// Error
{ error: { code: string, message: string, field?: string }, meta: { requestId:
string, timestamp: string } }
```

HTTP codes: 400 validation · 401 auth · 403 permission · 404 not found · 409 conflict/idempotency · 422 domain rejection · 429 rate limit · 5xx server/integration failure. Never return 200 containing an error.

---

## RBAC FORMAT

All permissions use dot notation: `domain.resource.action`

Examples: `finance.invoice.post` · `crm.lead.convert` · `hpay.payment.approve` · `procurement.po.approve`

---

## MODULE 01 — FINANCE CORE (AR / AP / GL)

**Band:** S2 (Domain API present) approaching S3

**Owner Domain:** Finance

**Status:** Partial — Journal entry model present; AR/AP lifecycle not complete

---

### M01.1 — POSITION IN ORGANISM

Finance Core is the financial system of record for Harvics OS. Every revenue and cost event in the system produces a journal entry here. AR receives invoices from CRM. AP receives bills from Procurement. GL is the ledger of record. All settlement flows through HPay (M03) and posts back here.

None

`CRM Order confirmed → Finance AR (invoice) → HPay payment → GL posted`  
`Procurement PO confirmed → Finance AP (bill) → HPay payment → GL posted`

Finance never originates commercial transactions. It records the financial consequence of commercial events.

---

### M01.2 — HONEST STATE (August 2026)

None

#### IMPLEMENTED

- ✓ Basic Prisma models exist (partial – verify schema)
- ✓ JWT auth on API routes
- ✓ `/api/finance/*` base routing

#### SCAFFOLD / UNKNOWN

- ~ Journal entry model – UNKNOWN if double-entry enforced
- ~ Period management – UNKNOWN
- ~ Chart of accounts – UNKNOWN

NOT BUILT

- X AR invoice lifecycle (draft → posted → paid → reconciled)
- X AP bill lifecycle (received → matched → approved → paid)
- X GL period close with trial balance
- X Multi-currency revaluation
- X Tax posting with M04 Tax Engine
- X Intercompany eliminations
- X Financial reporting (P&L, Balance Sheet, Cash Flow)
- X Data Ocean Bronze ingestion of journal entries

---

## M01.3 — ENTITY MODEL

### M01.3.1 ChartOfAccounts

None

```
model Account {
  id          String      @id @default(uuid())
  tenantId    String
  code        String      // e.g. 1100, 4000 – unique per tenant
  name        String
  type        AccountType // ASSET | LIABILITY | EQUITY | REVENUE | EXPENSE
  subtype     String?     // e.g. CURRENT_ASSET | FIXED_ASSET | COGS | OPEX
  currencyCode String     @default("USD")
  isActive    Boolean     @default(true)
  isSystem    Boolean     @default(false) // system accounts cannot be deleted
  parentId    String?     // hierarchical chart of accounts
  description  String?
  version     Int         @default(1)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  createdBy   String
  deletedAt   DateTime?

  parent      Account?    @relation("AccountHierarchy", fields: [parentId],
references: [id])
  children    Account[]   @relation("AccountHierarchy")
  debitLines  JournalLine[] @relation("DebitAccount")
  creditLines JournalLine[] @relation("CreditAccount")
}

enum AccountType {
```

```

    ASSET LIABILITY EQUITY REVENUE EXPENSE
}

```

### M01.3.2 FiscalPeriod

```

None
model FiscalPeriod {
    id          String          @id @default(uuid())
    tenantId    String
    name        String          // e.g. "Jan 2026"
    periodCode  String          // e.g. "2026-01"
    startDate   DateTime
    endDate     DateTime
    status      PeriodStatus    // OPEN | CLOSED | LOCKED
    closedBy    String?
    closedAt    DateTime?
    version     Int             @default(1)
    createdAt   DateTime        @default(now())
    updatedAt   DateTime        @updatedAt

    journals    JournalEntry[]
}

enum PeriodStatus {
    OPEN CLOSED LOCKED
}

```

### M01.3.3 JournalEntry (double-entry)

```

None
// RULE: sum(debit lines) MUST equal sum(credit lines) - enforced at write, not UI
model JournalEntry {
    id          String          @id @default(uuid())
    tenantId    String
    entryNumber  String          // system-generated, unique per tenant
    periodId    String
    journalDate  DateTime
    type        JournalType     // MANUAL | AR_INVOICE | AR_PAYMENT | AP_BILL |
    AP_PAYMENT | PAYROLL | ADJUSTMENT | FX_REVALUATION
    status      JournalStatus    // DRAFT | POSTED | REVERSED
    description  String
    reference    String?         // source document reference (invoice number, PO
    number, etc.)
    sourceModule String?         // CRM | PROCUREMENT | HPAY | PAYROLL | MANUAL
    sourceDocumentId String?     // foreign key to source document in its domain
}

```

```

    currencyCode      String
    exchangeRate      Decimal      @db.Decimal(18, 6) @default(1)
    totalDebit        BigInt        // minor units (e.g. cents) – always equals
totalCredit
    totalCredit       BigInt        // minor units
    postedBy          String?
    postedAt          DateTime?
    reversedBy        String?
    reversedAt        DateTime?
    reversalEntryId   String?
    idempotencyKey     String?      @unique
    version            Int          @default(1)
    createdAt         DateTime      @default(now())
    updatedAt         DateTime      @updatedAt
    createdBy         String

    period            FiscalPeriod @relation(fields: [periodId], references: [id])
    lines             JournalLine[]
}

model JournalLine {
    id                String          @id @default(uuid())
    journalId         String
    lineNumber        Int
    accountId         String          // Account.id
    description       String?
    debitAmount       BigInt          @default(0) // minor units; one of debit/credit
must be 0
    creditAmount      BigInt          @default(0) // minor units
    currencyCode      String
    taxCode           String?
    taxAmount         BigInt          @default(0)
    costCentre        String?
    projectCode       String?
    entityRef         String?        // e.g. customer ID, supplier ID for sub-ledger
tracing

    journal           JournalEntry @relation(fields: [journalId], references: [id])
    debitAccount      Account       @relation("DebitAccount", fields: [accountId],
references: [id])
}

enum JournalType {
    MANUAL AR_INVOICE AR_PAYMENT AP_BILL AP_PAYMENT PAYROLL ADJUSTMENT
    FX_REVALUATION
}

enum JournalStatus {

```

```
DRAFT POSTED REVERSED
}
```

## M01.3.4 ARInvoice

```
None
// STATUS: SPECIFIED – build after CRM SalesOrder is live
model ARInvoice {
  id          String          @id @default(uuid())
  tenantId    String
  invoiceNumber String        // system-generated
  customerId  String          // Customer.id (CRM)
  salesOrderId String?        // SalesOrder.id (CRM)
  periodId    String
  status      ARInvoiceStatus // DRAFT | POSTED | PART_PAID | PAID | OVERDUE |
WRITTEN_OFF | CANCELLED
  invoiceDate DateTime
  dueDate      DateTime
  currencyCode String
  subtotal     BigInt         // minor units
  taxAmount    BigInt
  totalAmount  BigInt
  paidAmount    BigInt        @default(0)
  outstandingAmount BigInt     // computed: totalAmount - paidAmount
  paymentTerms  String?
  notes         String?
  journalEntryId String?      // posted GL entry
  idempotencyKey String       @unique // prevent duplicate invoice from same
order
  version      Int            @default(1)
  createdAt    DateTime       @default(now())
  updatedAt    DateTime       @updatedAt
  createdBy    String
  deletedAt    DateTime?

  lines      ARInvoiceLine[]
  payments   ARPayment[]
}

model ARInvoiceLine {
  id          String    @id @default(uuid())
  invoiceId   String
  lineNumber  Int
  description String
  quantity    Decimal   @db.Decimal(18, 4)
  unitPrice   BigInt     // minor units
```

```

    taxCode      String?
    taxAmount    BigInt    @default(0)
    lineTotal    BigInt    // minor units
    accountId    String    // revenue account
    invoice      ARInvoice @relation(fields: [invoiceId], references: [id])
  }

model ARPayment {
  id            String      @id @default(uuid())
  tenantId      String
  invoiceId     String
  paymentDate   DateTime
  amount        BigInt      // minor units
  currencyCode  String
  method        String      // BANK_TRANSFER | HPAY | CHEQUE | CASH
  reference     String?     // bank reference or HPay transaction ID
  hpayTxId      String?
  journalEntryId String?
  idempotencyKey String      @unique
  version       Int         @default(1)
  createdAt     DateTime    @default(now())
  createdBy     String

  invoice       ARInvoice    @relation(fields: [invoiceId], references: [id])
}

enum ARInvoiceStatus {
  DRAFT POSTED PART_PAID PAID OVERDUE WRITTEN_OFF CANCELLED
}

```

### M01.3.5 APBill

```

None
// STATUS: SPECIFIED – build after Procurement PO is live
model APBill {
  id            String      @id @default(uuid())
  tenantId      String
  billNumber     String      // system-generated
  supplierId    String      // Supplier.id (Procurement)
  purchaseOrderId String?    // PurchaseOrder.id
  status        APBillStatus // RECEIVED | MATCHED | APPROVED | PAID | DISPUTED
| CANCELLED
  billDate       DateTime
  dueDate        DateTime
  currencyCode   String
  subtotal       BigInt

```

```

    taxAmount      BigInt
    totalAmount    BigInt
    paidAmount      BigInt      @default(0)
    approvedBy     String?
    approvedAt     DateTime?
    journalEntryId String?
    idempotencyKey String      @unique
    version        Int         @default(1)
    createdAt      DateTime    @default(now())
    updatedAt      DateTime    @updatedAt
    createdBy      String

    lines          APBillLine[]
    payments        APPayment[]
}

model APBillLine {
  id          String  @id @default(uuid())
  billId      String
  lineNumber  Int
  description  String
  quantity    Decimal @db.Decimal(18, 4)
  unitPrice   BigInt
  taxCode     String?
  taxAmount   BigInt  @default(0)
  lineTotal   BigInt
  accountId   String  // expense account
  bill        APBill  @relation(fields: [billId], references: [id])
}

model APPayment {
  id          String  @id @default(uuid())
  tenantId    String
  billId      String
  paymentDate DateTime
  amount      BigInt
  currencyCode String
  method      String
  hpayTxId    String?
  journalEntryId String?
  idempotencyKey String  @unique
  version     Int       @default(1)
  createdAt   DateTime  @default(now())
  createdBy   String

  bill        APBill  @relation(fields: [billId], references: [id])
}

```

```
enum APBillStatus {  
    RECEIVED MATCHED APPROVED PAID DISPUTED CANCELLED  
}
```

---

## M01.4 — KEY WORKFLOWS

### M01.4.1 POST AR INVOICE (triggered by CRM sales.order.confirmed event)

**Trigger:** Domain event `sales.order.confirmed` from CRM Module 05, OR typed API call `POST /api/finance/ar/invoices`

**Actor:** System (event-driven) | BillingClerk

**Idempotency:** `idempotencyKey = salesOrderId` — if invoice already exists for this order, return existing. Never duplicate.

#### Spine:

None

1. Auth + tenant isolation
2. Validate: SalesOrder exists, `status ∈ (CONFIRMED | IN_FULFILLMENT | SHIPPED | DELIVERED)`
3. Validate: no existing ARInvoice with same `idempotencyKey` (`salesOrderId`)
4. Resolve period: find OPEN FiscalPeriod covering invoice date – block if period CLOSED
5. Calculate: subtotal, taxAmount, totalAmount from order lines (re-compute, never trust submitted total)
6. Governance: Budget check (is amount within billing authority?) + Compliance (customer active, not on hold)
7. BEGIN TRANSACTION:
  - a. INSERT ARInvoice (`status = DRAFT`)
  - b. INSERT ARInvoiceLine per order line
  - c. INSERT JournalEntry (`status = DRAFT`):

|    |                            |             |
|----|----------------------------|-------------|
| DR | Accounts Receivable (1100) | totalAmount |
| CR | Revenue (4000)             | subtotal    |
| CR | Tax Payable (2200)         | taxAmount   |
  - d. UPDATE ARInvoice `status = POSTED`, `journalEntryId = $entryId`
  - e. UPDATE JournalEntry `status = POSTED`
  - f. INSERT AuditLog
8. COMMIT

```
9. Emit: invoice.created { invoiceId, customerId, salesOrderId, totalAmount,
dueDate, tenantId }
10. Notify: customer (external email via M20) + sales rep (in-app)
```

#### Failure modes:

- Period CLOSED → block, return 422 with explanation, notify Finance manager
- Tax Engine error → block invoice, return 503
- Idempotency hit → return existing invoice with 200

---

## M01.4.2 RECORD AR PAYMENT

**Trigger:** `payment.received` event from HPay (M03), OR `POST /api/finance/ar/invoices/:id/payments`

None

```
1. Auth + validate invoice exists and belongs to tenant
2. Validate: invoice status ∈ (POSTED | PART_PAID | OVERDUE) – cannot pay PAID or
CANCELLED
3. Validate: amount ≤ outstandingAmount (cannot overpay without credit note)
4. Idempotency check: hpayTxId or idempotencyKey
5. Governance: Compliance check (payment source legitimate?)
6. BEGIN TRANSACTION:
  a. INSERT ARPayment
  b. INSERT JournalEntry:
    DR Bank / Clearing (1010)    amount
    CR Accounts Receivable (1100) amount
  c. UPDATE ARInvoice.paidAmount += amount
  d. UPDATE ARInvoice.status = (paidAmount >= totalAmount) ? PAID : PART_PAID
  e. UPDATE ARInvoice.outstandingAmount = totalAmount - paidAmount
  f. INSERT AuditLog
7. COMMIT
8. Emit: ar.payment.recorded { invoiceId, paymentId, amount, outstandingAmount }
9. If PAID: emit invoice.paid { invoiceId, customerId, salesOrderId }
   → CRM consumes to update SalesOrder status
```

### M01.4.3 PERIOD CLOSE

**Actor:** FinanceManager | CFO

**Pre-conditions:** All journals for period are POSTED. No DRAFT journals remain.

- None
1. Governance: all five checks – CFO authority required for period close

2. Generate trial balance: SUM(debit) and SUM(credit) per account for period

3. Validate: trial balance is zero-balanced (totalDebit = totalCredit)

4. If not balanced: BLOCK close, list unbalanced accounts, require manual correction

5. If balanced:

a. UPDATE FiscalPeriod.status = CLOSED

b. For year-end: post closing entries (Revenue/Expense → Retained Earnings)

c. INSERT AuditLog with trial balance summary

6. Emit: period.closed { periodId, periodCode, trialBalance }

### M01.5 — API CONTRACT

None

| METHOD    | PATH                           | PERMISSION             | STATUS |
|-----------|--------------------------------|------------------------|--------|
| ACCOUNTS  |                                |                        |        |
| GET       | /api/finance/accounts          | finance.account.read   |        |
| SPECIFIED |                                |                        |        |
| POST      | /api/finance/accounts          | finance.account.manage |        |
| SPECIFIED |                                |                        |        |
| PATCH     | /api/finance/accounts/:id      | finance.account.manage |        |
| SPECIFIED |                                |                        |        |
| PERIODS   |                                |                        |        |
| GET       | /api/finance/periods           | finance.period.read    |        |
| SPECIFIED |                                |                        |        |
| POST      | /api/finance/periods           | finance.period.manage  |        |
| SPECIFIED |                                |                        |        |
| POST      | /api/finance/periods/:id/close | finance.period.close   |        |
| SPECIFIED |                                |                        |        |
| JOURNALS  |                                |                        |        |
| POST      | /api/finance/journals          | finance.journal.create |        |
| SPECIFIED |                                |                        |        |

|             |                                       |                           |
|-------------|---------------------------------------|---------------------------|
| GET         | /api/finance/journals                 | finance.journal.read      |
| SPECIFIED   |                                       |                           |
| GET         | /api/finance/journals/:id             | finance.journal.read      |
| SPECIFIED   |                                       |                           |
| POST        | /api/finance/journals/:id/post        | finance.journal.post      |
| SPECIFIED   |                                       |                           |
| POST        | /api/finance/journals/:id/reverse     | finance.journal.reverse   |
| SPECIFIED   |                                       |                           |
| AR INVOICES |                                       |                           |
| POST        | /api/finance/ar/invoices              | finance.ar.invoice.create |
| SPECIFIED   |                                       |                           |
| GET         | /api/finance/ar/invoices              | finance.ar.invoice.read   |
| SPECIFIED   |                                       |                           |
| GET         | /api/finance/ar/invoices/:id          | finance.ar.invoice.read   |
| SPECIFIED   |                                       |                           |
| POST        | /api/finance/ar/invoices/:id/payments | finance.ar.payment.record |
| SPECIFIED   |                                       |                           |
| GET         | /api/finance/ar/aging                 | finance.ar.report         |
| SPECIFIED   |                                       |                           |
| AP BILLS    |                                       |                           |
| POST        | /api/finance/ap/bills                 | finance.ap.bill.create    |
| SPECIFIED   |                                       |                           |
| GET         | /api/finance/ap/bills                 | finance.ap.bill.read      |
| SPECIFIED   |                                       |                           |
| GET         | /api/finance/ap/bills/:id             | finance.ap.bill.read      |
| SPECIFIED   |                                       |                           |
| POST        | /api/finance/ap/bills/:id/approve     | finance.ap.bill.approve   |
| SPECIFIED   |                                       |                           |
| POST        | /api/finance/ap/bills/:id/payments    | finance.ap.payment.create |
| SPECIFIED   |                                       |                           |
| REPORTING   |                                       |                           |
| GET         | /api/finance/reports/trial-balance    | finance.report.read       |
| SPECIFIED   |                                       |                           |
| GET         | /api/finance/reports/pl               | finance.report.read       |
| SPECIFIED   |                                       |                           |
| GET         | /api/finance/reports/balance-sheet    | finance.report.read       |
| SPECIFIED   |                                       |                           |
| GET         | /api/finance/reports/cash-flow        | finance.report.read       |
| SPECIFIED   |                                       |                           |
| GET         | /api/finance/reports/ar-aging         | finance.report.read       |
| SPECIFIED   |                                       |                           |
| GET         | /api/finance/reports/ap-aging         | finance.report.read       |
| SPECIFIED   |                                       |                           |

---

## M01.6 — RBAC

None

|                           |                                            |
|---------------------------|--------------------------------------------|
| finance.account.read      | - view chart of accounts                   |
| finance.account.manage    | - create/edit accounts                     |
| finance.journal.read      | - view journal entries                     |
| finance.journal.create    | - create manual journal entries            |
| finance.journal.post      | - post draft journals to GL                |
| finance.journal.reverse   | - reverse posted journals                  |
| finance.period.read       | - view fiscal periods                      |
| finance.period.manage     | - create fiscal periods                    |
| finance.period.close      | - close a period (CFO/FinanceManager only) |
| finance.ar.invoice.read   | - view AR invoices                         |
| finance.ar.invoice.create | - create/post invoices                     |
| finance.ar.payment.record | - record AR payments                       |
| finance.ap.bill.read      | - view AP bills                            |
| finance.ap.bill.create    | - enter supplier bills                     |
| finance.ap.bill.approve   | - approve bills for payment                |
| finance.ap.payment.create | - initiate AP payments (triggers HPay)     |
| finance.report.read       | - view financial reports                   |
| finance.report.export     | - export financial data                    |

Role assignments:

None

|                |                                                         |
|----------------|---------------------------------------------------------|
| CFO            | - all finance permissions                               |
| FinanceManager | - all except period.close, report.export needs approval |
| Accountant     | - journal.create/read, ar/ap read, invoice.create       |
| BillingClerk   | - ar.invoice.create, ar.payment.record                  |
| APClerk        | - ap.bill.create, ap.bill.read                          |
| Auditor        | - all .read permissions, no write                       |

---

## M01.7 — GRADUATION ROADMAP

None

| FEATURE                  | CURRENT    | TARGET | PRIORITY         |
|--------------------------|------------|--------|------------------|
| Chart of Accounts        | S1/UNKNOWN | S4     | Sprint 1 Finance |
| Fiscal Period management | S1/UNKNOWN | S4     | Sprint 1 Finance |

|                              |         |        |                                 |
|------------------------------|---------|--------|---------------------------------|
| Journal Entry (double-entry) | UNKNOWN | S4     | Sprint 1 Finance – BLOCKER      |
| AR Invoice lifecycle         | S0      | S4     | Sprint 2 – after CRM Order      |
| AR Payment recording         | S0      | S4     | Sprint 2                        |
| AP Bill lifecycle            | S0      | S4     | Sprint 3 – after Procurement PO |
| AP Payment                   | S0      | S4     | Sprint 3                        |
| Period Close with TB         | S0      | S4     | Sprint 4                        |
| Financial Reports            | S0      | S3     | Sprint 4                        |
| FX Revaluation               | S0      | TARGET | Post Sprint 4                   |
| Intercompany                 | S0      | TARGET | H2                              |
| Data Ocean integration       | S0      | TARGET | H2                              |

### Sprint sequence:

1. Verify/build Chart of Accounts + Fiscal Period Prisma models
2. Enforce double-entry constraint at application layer (not DB constraint alone)
3. Build AR invoice from CRM event (sales.order.confirmed consumer)
4. Build AR payment from HPay event (payment.received consumer)
5. Build AP bill from Procurement event (po.confirmed consumer)
6. Build period close with trial balance validation

## M01.8 — PROHIBITIONS

None

- X Never post a journal where `debitTotal ≠ creditTotal` – enforce server-side before insert
- X Never write to a CLOSED or LOCKED period – enforce server-side
- X Never duplicate an invoice for the same SalesOrder – `idempotencyKey = salesOrderId`
- X Never hard-delete journal entries or invoices – soft delete only, append-only for GL
- X Never allow Finance to write Customer or SalesOrder state – read only via CRM API
- X Never use Decimal for monetary values – BigInt minor units only
- X Never trust client-submitted invoice totals – recompute from lines server-side
- X Never estimate tax – call M04 Tax Engine; block if unavailable

# MODULE 03 — HPAY (PAYMENT RAIL)

**Band:** S1 (scaffold) — HPay conceptually defined, payment execution not live

**Owner Domain:** Payments

**Status:** Architecture specified; production payment rail not implemented

---

## M03.1 — POSITION IN ORGANISM

HPay is Harvics' internal payment execution rail. It processes outbound payments (AP settlements, payroll) and records inbound payments (customer collections). It does not hold customer balances — it routes, executes, and reconciles. All financial posting is done back in Finance Core (M01) via journal entries triggered by HPay events.

None

Finance AP approval → HPay payment instruction → Bank/PSP execution → payment.completed → Finance GL post  
Finance AR awaiting → HPay collection → payment.received → Finance AR update

HPay never writes to Finance. Finance never writes to HPay. They communicate through domain events only.

---

## M03.2 — HONEST STATE (August 2026)

None

### IMPLEMENTED

✗ Nothing confirmed implemented in HPay production rail

### SPECIFIED

~ HPay module architecture defined  
~ Payment model in Prisma (UNKNOWN – verify schema)

### NOT BUILT

✗ Payment instruction lifecycle  
✗ Bank/PSP adapter layer  
✗ Multi-currency settlement  
✗ Payment reconciliation  
✗ Fraud / velocity checks

- X Payment approval workflow (four-eyes for high value)
- X Outbound SWIFT / SEPA / local rails
- X Inbound payment matching

## M03.3 — ENTITY MODEL

None

```
model PaymentInstruction {
  id                String                @id @default(uuid())
  tenantId          String
  reference          String              // system-generated: PAY-YYYYMMDD-SEQ
  type              PaymentType          // SUPPLIER | CUSTOMER_REFUND | PAYROLL |
INTERCO | FX_PURCHASE
  direction          PaymentDirection    // OUTBOUND | INBOUND
  status             PaymentStatus        // DRAFT | PENDING_APPROVAL | APPROVED |
SUBMITTED | PROCESSING | COMPLETED | FAILED | CANCELLED | REVERSED
  amount            BigInt                // minor units
  currencyCode       String
  settlementAmount   BigInt?              // if converted, settlement currency amount
  settlementCurrency String?
  fxRate             Decimal?             @db.Decimal(18, 6)
  sourceAccountId    String?              // internal bank account or HPay wallet
  beneficiaryId      String?              // Beneficiary.id
  beneficiaryName     String              // denormalised for audit
  beneficiaryIban     String?
  beneficiarySwift    String?
  beneficiaryBank     String?
  paymentRail        String?              // SWIFT | SEPA | FASTER_PAYMENTS | LOCAL |
INTERNAL
  valueDate          DateTime?
  executedAt         DateTime?
  pspReference        String?              // external PSP transaction ID
  failureReason       String?
  reversalRef         String?
  sourceModule        String              // FINANCE_AP | FINANCE_AR | PAYROLL |
MANUAL
  sourceDocumentId   String              // APBill.id, ARInvoice.id, PayrollRun.id
  approvedBy         String?
  approvedAt         DateTime?
  idempotencyKey      String              @unique
  version            Int                  @default(1)
  createdAt          DateTime             @default(now())
  updatedAt          DateTime             @updatedAt
}
```

```

    createdBy      String
}

model Beneficiary {
  id              String      @id @default(uuid())
  tenantId       String
  name           String
  type           String      // SUPPLIER | EMPLOYEE | CUSTOMER | INTERCO
  entityId       String      // Supplier.id | Employee.id | Customer.id
  currencyCode   String
  iban           String?
  accountNumber  String?
  sortCode       String?
  swiftBic       String?
  bankName       String?
  bankCountry    String
  isVerified     Boolean      @default(false)
  verifiedBy     String?
  verifiedAt     DateTime?
  version        Int          @default(1)
  createdAt      DateTime      @default(now())
  updatedAt      DateTime      @updatedAt
  createdBy      String
  deletedAt      DateTime?
}

enum PaymentType {
  SUPPLIER CUSTOMER_REFUND PAYROLL INTERCO FX_PURCHASE
}

enum PaymentDirection {
  OUTBOUND INBOUND
}

enum PaymentStatus {
  DRAFT PENDING_APPROVAL APPROVED SUBMITTED PROCESSING COMPLETED FAILED CANCELLED
  REVERSED
}

```

---

## M03.4 — KEY WORKFLOWS

### M03.4.1 INITIATE OUTBOUND PAYMENT (AP Settlement)

**Trigger:** Finance AP bill approved → `POST /api/hpay/payments` (called by Finance module)

**Idempotency key:** `sourceDocumentId + paymentDate` — never duplicate a payment for the same bill

None

1. Auth: Finance service account with `hpay.payment.create` permission
2. Validate: bill exists, `status = APPROVED`, amount matches bill outstanding
3. Validate: beneficiary verified (`Beneficiary.isVerified = true`) – block unverified
4. Idempotency: check existing payment for same `idempotencyKey`
5. Governance: ALL FIVE CHECKS
  - Budget: payment within delegated authority?
  - Legal: destination country not sanctioned?
  - Compliance: AML check on beneficiary
  - Security: actor permitted?
  - Contract: payment terms consistent with PO?
6. If `amount > HIGH_VALUE_THRESHOLD`: require second approver (four-eyes)
7. BEGIN TRANSACTION:
  - a. INSERT `PaymentInstruction` (`status = PENDING_APPROVAL` or `APPROVED` if auto-approved)
  - b. INSERT `AuditLog`
8. COMMIT
9. If `APPROVED`: submit to PSP adapter (async job)
10. Emit: `hpay.payment.submitted { paymentId, amount, currency, beneficiaryId }`

### M03.4.2 PAYMENT COMPLETION (PSP callback)

None

1. PSP webhook → `POST /api/hpay/webhook` (verified signature required)
2. Lookup `PaymentInstruction` by `pspReference`
3. BEGIN TRANSACTION:
  - a. UPDATE `PaymentInstruction.status = COMPLETED`, `executedAt = NOW()`, `pspReference = $ref`
  - b. INSERT `AuditLog`
4. COMMIT
5. Emit: `payment.completed { paymentId, sourceModule, sourceDocumentId, amount, currency }`
  - Finance consumes: post GL entry DR AP / CR Bank
  - Procurement consumes: update PO payment status

---

## M03.5 — API CONTRACT

| None   |                                    |                          |           |
|--------|------------------------------------|--------------------------|-----------|
| METHOD | PATH                               | PERMISSION               | STATUS    |
| POST   | /api/hpay/payments                 | hpay.payment.create      | SPECIFIED |
| GET    | /api/hpay/payments                 | hpay.payment.read        | SPECIFIED |
| GET    | /api/hpay/payments/:id             | hpay.payment.read        | SPECIFIED |
| POST   | /api/hpay/payments/:id/approve     | hpay.payment.approve     | SPECIFIED |
| POST   | /api/hpay/payments/:id/cancel      | hpay.payment.cancel      | SPECIFIED |
| POST   | /api/hpay/payments/:id/reverse     | hpay.payment.reverse     | SPECIFIED |
| GET    | /api/hpay/beneficiaries            | hpay.beneficiary.read    | SPECIFIED |
| POST   | /api/hpay/beneficiaries            | hpay.beneficiary.manage  | SPECIFIED |
| POST   | /api/hpay/beneficiaries/:id/verify | hpay.beneficiary.verify  | SPECIFIED |
| POST   | /api/hpay/webhook                  | INTERNAL (PSP only)      | SPECIFIED |
| GET    | /api/hpay/reconciliation           | hpay.reconciliation.read | SPECIFIED |

---

## M03.6 — RBAC

|                          |                                                               |
|--------------------------|---------------------------------------------------------------|
| None                     |                                                               |
| hpay.payment.read        | - view payment instructions                                   |
| hpay.payment.create      | - initiate payments (Finance AP service or Finance staff)     |
| hpay.payment.approve     | - approve payments (FinanceManager / CFO depending on amount) |
| hpay.payment.cancel      | - cancel pending payments                                     |
| hpay.payment.reverse     | - reverse completed payments (CFO only)                       |
| hpay.beneficiary.read    | - view beneficiaries                                          |
| hpay.beneficiary.manage  | - add/edit beneficiaries                                      |
| hpay.beneficiary.verify  | - mark beneficiary as bank-verified                           |
| hpay.reconciliation.read | - view reconciliation reports                                 |

---

## M03.7 — GRADUATION ROADMAP

None

| FEATURE                          | CURRENT | TARGET | PRIORITY                         |
|----------------------------------|---------|--------|----------------------------------|
| PaymentInstruction Prisma model  | UNKNOWN | S4     | Sprint 1 Finance                 |
| Beneficiary model + verification | UNKNOWN | S4     | Sprint 1 Finance                 |
| Outbound payment workflow        | S0      | S4     | Sprint 3 (after AP Bill)         |
| Four-eyes approval               | S0      | S4     | Sprint 3                         |
| PSP adapter (Stripe/Banking API) | S0      | S4     | Sprint 4 – requires PSP contract |
| Inbound payment matching         | S0      | S4     | Sprint 4                         |
| Reconciliation                   | S0      | S3     | Sprint 4                         |
| FX purchase                      | S0      | TARGET | H2                               |
| Multi-rail (SWIFT/SEPA)          | S0      | TARGET | H2                               |

## M03.8 — PROHIBITIONS

None

- ✗ Never store card numbers, CVVs, or full bank credentials in HPay tables – use PSP tokens only
- ✗ Never auto-approve payments above HIGH\_VALUE\_THRESHOLD – four-eyes is mandatory
- ✗ Never process payment to unverified beneficiary
- ✗ Never duplicate a payment – idempotencyKey is mandatory
- ✗ Never post GL entries from HPay – emit event, Finance posts
- ✗ Never expose PSP credentials in API responses or logs

## MODULE 04 — TAX ENGINE

**Band:** S0 — not built; required for Quote and Invoice

**Owner Domain:** Finance (cross-cutting service)

**Status:** Architecture defined; no implementation

## M04.1 — POSITION IN ORGANISM

Tax Engine is a synchronous service called by CRM (Quote), Finance (Invoice), and Procurement (Bill). It returns the correct tax rate and amount for a given product + jurisdiction. It does not store business transactions — it is a calculation service. Tax postings live in Finance Core journals.

**Rule:** Modules MUST NOT estimate tax. If Tax Engine is unavailable, Quote/Invoice creation must block with 503.

---

## M04.2 — ENTITY MODEL

None

```
model TaxJurisdiction {
  id          String      @id @default(uuid())
  tenantId    String
  countryCode  String      // ISO 3166-1 alpha-2
  stateCode   String?     // for US state tax
  name        String      // e.g. "UAE VAT", "UK VAT Standard"
  code        String      // e.g. "AE-VAT", "GB-VAT-STD"
  type        String      // VAT | GST | SALES_TAX | EXCISE | WITHHOLDING
  rate        Decimal     @db.Decimal(7, 6) // e.g. 0.050000 = 5%
  effectiveFrom DateTime
  effectiveTo  DateTime?
  isActive    Boolean     @default(true)
  version      Int        @default(1)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  createdBy   String
}
```

```
model TaxCode {
  id          String      @id @default(uuid())
  tenantId    String
  code        String      // e.g. T1, EXEMPT, ZERO
  name        String
  jurisdictionId String
  hsCode      String?     // for trade/customs
  isExempt    Boolean     @default(false)
  isZeroRated Boolean     @default(false)
  version      Int        @default(1)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
}
```

```

}

model TaxCalculationLog {
  id          String    @id @default(uuid())
  tenantId    String
  requestedAt DateTime  @default(now())
  sourceModule String    // CRM | FINANCE | PROCUREMENT
  sourceDocId String
  taxCode     String
  jurisdictionId String
  grossAmount BigInt
  taxRate     Decimal    @db.Decimal(7, 6)
  taxAmount   BigInt
  currencyCode String
}

```

## M04.3 — API CONTRACT

| None   |                        |               |           |
|--------|------------------------|---------------|-----------|
| METHOD | PATH                   | PERMISSION    | STATUS    |
| POST   | /api/tax/calculate     | tax.calculate | SPECIFIED |
| GET    | /api/tax/jurisdictions | tax.read      | SPECIFIED |
| POST   | /api/tax/jurisdictions | tax.manage    | SPECIFIED |
| GET    | /api/tax/codes         | tax.read      | SPECIFIED |
| POST   | /api/tax/codes         | tax.manage    | SPECIFIED |

### Calculate request:

```

TypeScript
interface TaxCalculateRequest {
  tenantId: string
  countryCode: string
  stateCode?: string
  taxCode: string
  grossAmount: number // BigInt minor units
  currencyCode: string
  transactionDate: string
}

interface TaxCalculateResponse {

```

```

taxCode: string
jurisdictionId: string
rate: number
taxAmount: number    // BigInt minor units
isExempt: boolean
isZeroRated: boolean
calculationRef: string // log ID for audit
}

```

**Failure behaviour:** If Tax Engine throws 5xx, calling module must return 503 to its client. Never substitute a default rate. Never estimate.

---

## M04.4 — GRADUATION ROADMAP

None

| FEATURE                    | CURRENT | TARGET | PRIORITY                     |
|----------------------------|---------|--------|------------------------------|
| TaxJurisdiction model      | S0      | S4     | Sprint 2 (blocker for Quote) |
| TaxCode model              | S0      | S4     | Sprint 2                     |
| Calculate API              | S0      | S4     | Sprint 2                     |
| UAE VAT (5%)               | S0      | S4     | Sprint 2 – primary market    |
| UK VAT (20%)               | S0      | S4     | Sprint 2                     |
| Zero-rated / Exempt logic  | S0      | S4     | Sprint 2                     |
| HS code / customs tax      | S0      | TARGET | Sprint 4                     |
| Multi-jurisdiction cascade | S0      | TARGET | H2                           |

---

## MODULE 05 — CRM + SALES (AMENDED)

Full specification is in **05-CRM-SALES\_MASTER.md**  
(HARVICS\_FINAL\_CURSOR\_MASTER package).

This section records amendments and corrections to that file.

---

## M05 AMENDMENTS (August 2026)

### Amendment 1 — RBAC dot notation

All permissions in the CRM spec use colon notation (`crm:lead:create`). The system RBAC standard is dot notation (`crm.lead.create`). **Pending architectural decision.** Until resolved, colon notation is preserved in the module spec. Do not mix formats in the same middleware.

### Amendment 2 — Monetary fields

All `Decimal @db.Decimal(18,4)` monetary fields are flagged for migration to `BigInt` minor units as the system standard matures. The current CRM Prisma models use `Decimal` — do not change without a formal migration plan and Finance sign-off.

### Amendment 3 — version field

Add `version Int @default(1)` to: Customer, Lead, Deal, Quote, SalesOrder, CrmActivity, CreditLimit. Used for optimistic locking. Client must submit `version` on PATCH; server rejects if version mismatch (409 Conflict).

### Amendment 4 — Order Management boundary

`SalesOrder` is the commercial commitment. Fulfillment execution (pick/pack/ship) belongs to Logistics (M13) and WMS (M10). CRM releases the order; it does not manage warehouse operations. The `SalesOrder.status` field reflects commercial state, not operational fulfillment state.

### Amendment 5 — Rebates boundary

`Rebate` entity is temporarily housed in CRM for MVP. Target architecture: move to a Trade Promotion Management (TPM) domain post-Sprint 4. Do not build complex rebate calculation logic inside CRM — keep the entity simple for now.

### Amendment 6 — Gate 5 (Production)

Add to the graduation model:

| Stage | Meaning          |
|-------|------------------|
| S0    | Catalogue        |
| S1    | Generic scaffold |

| Stage         | Meaning                                                                   |
|---------------|---------------------------------------------------------------------------|
| S2            | Domain API                                                                |
| S3            | Operator UI with real data                                                |
| S4            | Controlled: auth, audit, governance, RBAC, tested                         |
| <b>Gate 5</b> | <b>Production: live customer data, SLA, monitoring, incident response</b> |

Gate 5 is the commercial go-live gate. S4 is internal readiness. Gate 5 requires: production load test, DR plan tested, security penetration test, customer data handling review, on-call runbook.

---

# MODULE 06 — MARKETING + HARVYX

**Band:** S1 (HarvyX outreach exists; Marketing OS module is scaffold)  
**Owner Domain:** Marketing  
**Status:** HarvyX is a separate outreach tool feeding CRM leads; Marketing module not built

---

## M06.1 — POSITION IN ORGANISM

Marketing owns the top of the commercial funnel. HarvyX is the outbound AI-powered outreach engine. It generates prospect contacts and sequences outbound communications. Its output — engaged prospects — is imported into CRM Module 05 as Leads. Marketing does not own Leads after import.

None

HarvyX Prospect → outreach sequence → engagement signal → Lead import job → CRM  
Lead.status = NEW  
CRM Lead.qualified → lead.qualified event → Marketing (journey exit / attribution update)

## M06.2 — ENTITY MODEL

None

```
model Campaign {
  id          String          @id @default(uuid())
  tenantId    String
  name        String
  type        CampaignType    // EMAIL | HARVYX_SEQUENCE | EVENT | DIGITAL |
TRADE_SHOW | WHATSAPP
  status      CampaignStatus  // DRAFT | ACTIVE | PAUSED | COMPLETED | CANCELLED
  startDate   DateTime?
  endDate     DateTime?
  budget      BigInt?         // minor units
  spend       BigInt          @default(0)
  targetSegment String?
  ownerId     String          // User.id
  version     Int             @default(1)
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt
  createdBy   String
  deletedAt   DateTime?

  touches     CampaignTouch[]
  leads       CampaignLead[]
}
```

```
model CampaignTouch {
  id          String          @id @default(uuid())
  tenantId    String
  campaignId   String
  channel      String         // EMAIL | WHATSAPP | CALL | AD
  sentAt       DateTime?
  openedAt     DateTime?
  clickedAt    DateTime?
  respondedAt   DateTime?
  prospectEmail String?
  prospectId   String?       // HarveyX prospect ID
  createdAt    DateTime       @default(now())
  campaign     Campaign       @relation(fields: [campaignId], references: [id])
}
```

```
model CampaignLead {
  id          String          @id @default(uuid())
  campaignId   String
  leadId       String         // CRM Lead.id
  sourceTouch  String?        // CampaignTouch.id - attribution
  importedAt   DateTime       @default(now())
}
```

```

    campaign    Campaign @relation(fields: [campaignId], references: [id])
  }

model HarvyXImportJob {
  id            String      @id @default(uuid())
  tenantId      String
  campaignId    String?
  status        String      // PENDING | RUNNING | COMPLETED | FAILED
  importedCount Int          @default(0)
  failedCount   Int          @default(0)
  lastRunAt     DateTime?
  errorLog      String?
  createdAt     DateTime    @default(now())
  updatedAt     DateTime    @updatedAt
}

enum CampaignType {
  EMAIL HARVYX_SEQUENCE EVENT DIGITAL TRADE_SHOW WHATSAPP
}

enum CampaignStatus {
  DRAFT ACTIVE PAUSED COMPLETED CANCELLED
}

```

---

## M06.3 — KEY WORKFLOWS

### M06.3.1 HARVYX LEAD IMPORT JOB

**Trigger:** Scheduled job (daily or on-demand) OR `POST /api/marketing/harvyx/import`  
**Idempotency:** `externalId` on Lead — if Lead with this HarvyX ID already exists, skip. Never duplicate.

None

1. Auth: SystemImport service account (marketing.import permission)
2. Fetch new prospects from HarvyX API since last successful import
3. For each prospect:
  - a. Check Lead table: exists with same externalId? → skip (idempotent)
  - b. Validate prospect data (email format, name not empty)
  - c. INSERT Lead { source = HARVYX, status = NEW, externalId = harvyxId, ... }
  - d. If campaign attached: INSERT CampaignLead { campaignId, leadId }
  - e. Emit: lead.created (CRM handles AI scoring async)
4. UPDATE HarvyXImportJob: importedCount, failedCount, status = COMPLETED

5. Notify: Marketing manager – import summary

## M06.3.2 CAMPAIGN PERFORMANCE REPORTING

**Source:** Join Campaign + CampaignTouch + CampaignLead + CRM Lead (qualified/converted counts)

**Rule:** Never fabricate conversion rates. If CRM data is unavailable, show null — not zero.

None

Campaign metrics:

- Touches sent / opened / clicked / responded
- Leads generated (count of CampaignLead)
- Leads qualified (CRM Lead.status = QUALIFIED, linked to campaign)
- Leads converted (CRM Lead.status = CONVERTED)
- Revenue attributed (linked SalesOrder.totalAmount)
- Cost per lead (campaign.spend / leadsGenerated)
- ROI (attributedRevenue - spend / spend)

## M06.4 — API CONTRACT

None

| METHOD | PATH                               | PERMISSION                | STATUS    |
|--------|------------------------------------|---------------------------|-----------|
| GET    | /api/marketing/campaigns           | marketing.campaign.read   | SPECIFIED |
| POST   | /api/marketing/campaigns           | marketing.campaign.create | SPECIFIED |
| PATCH  | /api/marketing/campaigns/:id       | marketing.campaign.update | SPECIFIED |
| GET    | /api/marketing/campaigns/:id/stats | marketing.campaign.read   | SPECIFIED |
| POST   | /api/marketing/harvyx/import       | marketing.import          | SPECIFIED |
| GET    | /api/marketing/harvyx/jobs         | marketing.import          | SPECIFIED |
| GET    | /api/marketing/attribution         | marketing.report          | SPECIFIED |

## M06.5 — GRADUATION ROADMAP

None

| FEATURE | CURRENT | TARGET | PRIORITY |
|---------|---------|--------|----------|
|---------|---------|--------|----------|

|                            |    |    |                                      |
|----------------------------|----|----|--------------------------------------|
| Campaign Prisma model      | S0 | S4 | Sprint 2 Marketing                   |
| HarvyX import job          | S1 | S4 | Sprint 1 Marketing (blocker for CRM) |
| Campaign attribution       | S0 | S3 | Sprint 3 Marketing                   |
| Performance reporting      | S0 | S3 | Sprint 3 Marketing                   |
| Marketing UI /os/marketing | S0 | S3 | Sprint 2 Marketing                   |

## MODULE 07 — PROCUREMENT + SOURCE-TO-PAY

**Band:** S1 (scaffold) — Procurement module not built as domain product

**Owner Domain:** Procurement

**Status:** Architecture defined; no domain implementation

### M07.1 — POSITION IN ORGANISM

Procurement is the second primary commercial highway. It manages all inbound spend: from purchase request through PO issuance to goods receipt and supplier payment. The Source-to-Pay flow feeds Finance AP with bills and triggers HPay payments.

None

Purchase Requisition (approved) → Purchase Order → Goods Receipt → AP Bill → HPay Payment → GL

Procurement is also the origin of inventory replenishment signals to M10 (Inventory + WMS).

### M07.2 — HONEST STATE (August 2026)

None

NOT BUILT

- ✗ Purchase Requisition lifecycle
- ✗ Purchase Order with approval

- X Supplier master management
- X Goods Receipt Note (GRN)
- X Three-way match (PO → GRN → Invoice)
- X Supplier performance tracking
- X Procurement governance (budget authority matrix)
- X Source-to-Pay event stream

---

## M07.3 — ENTITY MODEL

### M07.3.1 Supplier

```
None
model Supplier {
  id                String                @id @default(uuid())
  tenantId          String
  code              String                // unique per tenant
  name              String
  legalName         String?
  type              SupplierType         // MANUFACTURER | DISTRIBUTOR | SERVICE |
CONTRACTOR | INTERCO
  status            SupplierStatus       // ACTIVE | INACTIVE | BLACKLISTED |
PENDING_APPROVAL
  countryCode       String
  currencyCode      String
  paymentTerms      String?              // NET30 | NET60 | CIA | etc.
  taxId             String?
  vatNumber         String?
  website           String?
  contactName       String?
  contactEmail      String?
  contactPhone      String?
  bankAccountId     String?              // HPay Beneficiary.id
  creditRating      String?
  qualifiedAt       DateTime?
  qualifiedBy       String?
  notes             String?
  tags              String[]
  version           Int                  @default(1)
  createdAt         DateTime             @default(now())
  updatedAt         DateTime             @updatedAt
  createdBy         String
  deletedAt         DateTime?
```

```

    purchaseOrders PurchaseOrder[]
}

enum SupplierType {
    MANUFACTURER DISTRIBUTOR SERVICE CONTRACTOR INTERCO
}

enum SupplierStatus {
    ACTIVE INACTIVE BLACKLISTED PENDING_APPROVAL
}

```

### M07.3.2 PurchaseRequisition

```

None
model PurchaseRequisition {
    id String @id @default(uuid())
    tenantId String
    prNumber String // system-generated: PR-YYYYMMDD-SEQ
    requestedBy String // User.id
    department String?
    status PRStatus // DRAFT | SUBMITTED | APPROVED | REJECTED |
    PO_CREATED | CANCELLED
    priority String // LOW | NORMAL | URGENT
    requiredByDate DateTime?
    estimatedValue BigInt // minor units
    currencyCode String
    justification String
    approvedBy String?
    approvedAt DateTime?
    rejectedBy String?
    rejectedReason String?
    governanceRef String?
    version Int @default(1)
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
    createdBy String

    lines PRLine[]
    purchaseOrders PurchaseOrder[]
}

model PRLine {
    id String @id @default(uuid())
    prId String
    lineNumber Int
}

```

```

    productId      String?    // Product catalogue ref (if stocked item)
    description     String     // free text if no product catalogue match
    quantity        Decimal    @db.Decimal(18, 4)
    uom             String
    estimatedUnit    BigInt     // minor units
    estimatedTotal   BigInt
    preferredSupplier String? // Supplier.id
    notes           String?
    pr              PurchaseRequisition @relation(fields: [prId], references: [id])
}

enum PRStatus {
    DRAFT SUBMITTED APPROVED REJECTED PO_CREATED CANCELLED
}

```

### M07.3.3 PurchaseOrder

```

None
model PurchaseOrder {
    id                String      @id @default(uuid())
    tenantId          String
    poNumber           String      // system-generated: PO-YYYYMMDD-SEQ
    prId              String?     // source PR (optional - can create PO without PR)
    supplierId        String
    status             POStatus   // DRAFT | SUBMITTED | ACKNOWLEDGED | PART_DELIVERED
    FULLY_DELIVERED | INVOICED | CLOSED | CANCELLED
    orderDate          DateTime
    requestedDelivery  DateTime?
    confirmedDelivery  DateTime?
    currencyCode       String
    subtotal           BigInt
    taxAmount          BigInt
    totalAmount        BigInt
    paymentTerms       String?
    deliveryAddress     Json?
    deliveryTerms      String?    // Incoterms
    warehouseId        String?
    supplierRef        String?    // supplier's own PO reference
    governanceRef      String?
    approvedBy         String?
    approvedAt         DateTime?
    idempotencyKey     String      @unique
    version            Int         @default(1)
    createdAt          DateTime   @default(now())
    updatedAt          DateTime   @updatedAt
    createdBy          String
}

```

```

    deletedAt      DateTime?

    supplier      Supplier    @relation(fields: [supplierId], references: [id])
    pr             PurchaseRequisition? @relation(fields: [prId], references: [id])
    lines          POLine[]
    grns           GoodsReceiptNote[]
}

model POLine {
  id              String    @id @default(uuid())
  poId            String
  lineNumber      Int
  productId       String?
  description     String
  quantity        Decimal @db.Decimal(18, 4)
  uom             String
  unitPrice       BigInt
  taxCode         String?
  taxAmount       BigInt   @default(0)
  lineTotal       BigInt
  receivedQty     Decimal @db.Decimal(18, 4) @default(0)
  invoicedQty     Decimal @db.Decimal(18, 4) @default(0)
  notes           String?
  po              PurchaseOrder @relation(fields: [poId], references: [id])
}

model GoodsReceiptNote {
  id              String    @id @default(uuid())
  tenantId        String
  grnNumber       String    // system-generated
  poId            String
  receivedAt      DateTime
  receivedBy      String    // User.id
  warehouseId     String?
  status          String    // DRAFT | CONFIRMED | DISPUTED
  notes           String?
  version         Int       @default(1)
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  po              PurchaseOrder @relation(fields: [poId], references: [id])
  lines           GRNLine[]
}

model GRNLine {
  id              String    @id @default(uuid())
  grnId           String
  poLineId        String

```

```

receivedQty Decimal @db.Decimal(18, 4)
acceptedQty Decimal @db.Decimal(18, 4)
rejectedQty Decimal @db.Decimal(18, 4) @default(0)
rejectionReason String?
grn          GoodsReceiptNote @relation(fields: [grnId], references: [id])
}

enum POStatus {
    DRAFT SUBMITTED ACKNOWLEDGED PART_DELIVERED FULLY_DELIVERED INVOICED CLOSED
    CANCELLED
}

```

## M07.4 — KEY WORKFLOWS

### M07.4.1 RAISE PURCHASE REQUISITION

**Trigger:** `POST /api/procurement/requisitions`

**Actor:** Any employee with `procurement.pr.create`

None

1. Auth + tenant isolation
2. Validate: lines not empty, `estimatedValue > 0`, `requiredByDate` in future
3. Governance: Budget check (does this PR exceed department budget?)
4. BEGIN TRANSACTION:
  - a. INSERT `PurchaseRequisition` (`status = DRAFT`)
  - b. INSERT `PRLine` per line
  - c. INSERT `AuditLog`
5. COMMIT
6. If auto-submit policy: submit immediately
7. Emit: `pr.created` { `prId`, `requestedBy`, `estimatedValue`, `currency` }
8. Notify: approver (in-app + email)

### M07.4.2 APPROVE PR → CREATE PO

**Trigger:** `POST /api/procurement/requisitions/:id/approve` + `POST /api/procurement/orders`

**Actor:** `ProcurementManager` | Director (above threshold)

None

1. Validate: PR `status = SUBMITTED`, actor has `procurement.pr.approve` permission

2. Governance: Budget authority check (is approver within authority for this amount?)
3. UPDATE PurchaseRequisition status = APPROVED
4. Emit: pr.approved

Then PO creation:

1. Validate: supplierId exists, status = ACTIVE
2. Validate: PO lines match PR lines (quantity, product)
3. Resolve tax: call Tax Engine per line – block if unavailable
4. Governance: ALL FIVE CHECKS (PO is a financial commitment)
  - Budget: within approved budget?
  - Legal: supplier country risk? Export restrictions?
  - Contract: existing framework contract with supplier? Pricing consistent?
  - Security: actor has procurement.po.create?
  - Compliance: supplier not blacklisted, AML check
5. BEGIN TRANSACTION:
  - a. INSERT PurchaseOrder (status = SUBMITTED)
  - b. INSERT POLine per line
  - c. UPDATE PR.status = PO\_CREATED
  - d. INSERT AuditLog
6. COMMIT
7. Emit: po.confirmed { poId, supplierId, totalAmount, currency, tenantId }
  - Finance AP creates draft bill
  - Inventory updates expected stock-in

## M07.4.3 THREE-WAY MATCH (GRN → PO → Invoice)

None

On GRN creation (goods physically received):

1. INSERT GoodsReceiptNote + GRNLines
2. UPDATE POLine.receivedQty += grnLine.acceptedQty
3. UPDATE PO.status = PART\_DELIVERED or FULLY\_DELIVERED
4. Emit: grn.confirmed { grnId, poId, receivedLines }
  - Inventory consumes: update stock-on-hand

On AP Bill receipt from supplier:

1. Three-way match: compare APBill lines vs POLine (price + qty) vs GRNLine (qty received)
  2. If match within tolerance: auto-approve for payment
  3. If mismatch: bill goes to DISPUTED status, notify ProcurementManager
  4. Once matched and approved: emit ap.bill.approved → HPay initiates payment
-

## M07.5 — API CONTRACT

| None               |                                           |                             |
|--------------------|-------------------------------------------|-----------------------------|
| METHOD             | PATH                                      | PERMISSION                  |
| STATUS             |                                           |                             |
| POST<br>SPECIFIED  | /api/procurement/suppliers                | procurement.supplier.create |
| GET<br>SPECIFIED   | /api/procurement/suppliers                | procurement.supplier.read   |
| GET<br>SPECIFIED   | /api/procurement/suppliers/:id            | procurement.supplier.read   |
| PATCH<br>SPECIFIED | /api/procurement/suppliers/:id            | procurement.supplier.update |
| POST<br>SPECIFIED  | /api/procurement/requisitions             | procurement.pr.create       |
| GET<br>SPECIFIED   | /api/procurement/requisitions             | procurement.pr.read         |
| GET<br>SPECIFIED   | /api/procurement/requisitions/:id         | procurement.pr.read         |
| POST<br>SPECIFIED  | /api/procurement/requisitions/:id/submit  | procurement.pr.submit       |
| POST<br>SPECIFIED  | /api/procurement/requisitions/:id/approve | procurement.pr.approve      |
| POST<br>SPECIFIED  | /api/procurement/requisitions/:id/reject  | procurement.pr.approve      |
| POST<br>SPECIFIED  | /api/procurement/orders                   | procurement.po.create       |
| GET<br>SPECIFIED   | /api/procurement/orders                   | procurement.po.read         |
| GET<br>SPECIFIED   | /api/procurement/orders/:id               | procurement.po.read         |
| POST<br>SPECIFIED  | /api/procurement/orders/:id/acknowledge   | procurement.po.update       |
| POST<br>SPECIFIED  | /api/procurement/orders/:id/cancel        | procurement.po.cancel       |
| POST<br>SPECIFIED  | /api/procurement/grn                      | procurement.grn.create      |
| GET<br>SPECIFIED   | /api/procurement/grn/:id                  | procurement.grn.read        |
| POST<br>SPECIFIED  | /api/procurement/grn/:id/confirm          | procurement.grn.confirm     |

```
GET      /api/procurement/three-way-match/:poId  procurement.po.read
SPECIFIED
```

---

## M07.6 — RBAC

None

|                              |                                          |
|------------------------------|------------------------------------------|
| procurement.supplier.read    | - view supplier master                   |
| procurement.supplier.create  | - add suppliers                          |
| procurement.supplier.update  | - edit suppliers                         |
| procurement.supplier.approve | - approve new supplier (compliance gate) |
| procurement.pr.create        | - raise purchase requisition             |
| procurement.pr.read          | - view requisitions                      |
| procurement.pr.submit        | - submit for approval                    |
| procurement.pr.approve       | - approve/reject PR (manager)            |
| procurement.po.create        | - create purchase order                  |
| procurement.po.read          | - view purchase orders                   |
| procurement.po.approve       | - approve PO above threshold             |
| procurement.po.cancel        | - cancel PO                              |
| procurement.po.update        | - update PO fields                       |
| procurement.grn.create       | - create goods receipt note              |
| procurement.grn.read         | - view GRNs                              |
| procurement.grn.confirm      | - confirm goods received                 |

Role assignments:

None

|                    |                                                         |
|--------------------|---------------------------------------------------------|
| Employee           | - pr.create, pr.read                                    |
| ProcurementOfficer | - all PR + PO create/read, grn.create/confirm           |
| ProcurementManager | - all above + pr.approve, po.approve (within authority) |
| Director           | - all + po.approve (high value), supplier.approve       |
| Finance            | - po.read, grn.read (for three-way match)               |
| Auditor            | - all .read permissions                                 |

---

# M07.7 — GRADUATION ROADMAP

None

| FEATURE                        | CURRENT | TARGET | PRIORITY                   |
|--------------------------------|---------|--------|----------------------------|
| Supplier Prisma model          | S0      | S4     | Sprint 1 Procurement       |
| PR lifecycle                   | S0      | S4     | Sprint 1 Procurement       |
| PO lifecycle with approval     | S0      | S4     | Sprint 2 Procurement       |
| GRN (goods receipt)            | S0      | S4     | Sprint 2 Procurement       |
| Three-way match                | S0      | S4     | Sprint 3 Procurement       |
| AP bill from GRN               | S0      | S4     | Sprint 3 (Finance M01 dep) |
| Supplier performance KPIs      | S0      | S3     | Sprint 4                   |
| Procurement UI /os/procurement | S0      | S3     | Sprint 2 Procurement       |

# MODULE 08 — LEGAL + COMPLIANCE

**Band:** S0 — not built  
**Owner Domain:** Legal  
**Status:** Governance checks reference legal/compliance data that does not yet exist as a domain

## M08.1 — POSITION IN ORGANISM

Legal + Compliance owns: contracts (commercial and supplier), regulatory compliance, KYC/AML screening, sanctions checks, export control (HS code restrictions). Neural Governance (M22) calls Legal for its Legal and Compliance checks. Without M08 data, those checks cannot be substantive.

## M08.2 — ENTITY MODEL

None

|                  |        |                      |
|------------------|--------|----------------------|
| model Contract { |        |                      |
| id               | String | @id @default(uuid()) |
| tenantId         | String |                      |
| contractNumber   | String | // system-generated  |

```

    type          ContractType    // CUSTOMER_FRAMEWORK | SUPPLIER | EMPLOYMENT |
NDA | SERVICE | REGULATORY
    status        ContractStatus  // DRAFT | UNDER_REVIEW | APPROVED | ACTIVE |
EXPIRED | TERMINATED
    counterpartyId String          // Customer.id or Supplier.id
    counterpartyType String        // CUSTOMER | SUPPLIER | EMPLOYEE
    title         String
    description    String?
    startDate      DateTime
    endDate        DateTime?
    autoRenewal    Boolean          @default(false)
    value          BigInt?          // contract value if fixed - minor units
    currencyCode   String?
    signedBy       String?          // User.id
    signedAt       DateTime?
    externalSigned Boolean          @default(false)
    documentVaultId String?         // signed PDF storage reference
    governanceRef  String?
    tags           String[]
    version        Int              @default(1)
    createdAt      DateTime          @default(now())
    updatedAt      DateTime          @updatedAt
    createdBy      String
    deletedAt      DateTime?

    clauses        ContractClause[]
}

```

```

model ContractClause {
    id          String    @id @default(uuid())
    contractId  String
    clauseNumber String
    type        String    // PAYMENT_TERMS | PRICE | SLA | EXCLUSIVITY | LIABILITY |
TERMINATION
    text        String
    isActive    Boolean    @default(true)
    contract    Contract   @relation(fields: [contractId], references: [id])
}

```

```

model ComplianceRecord {
    id          String    @id @default(uuid())
    tenantId    String
    entityId    String    // Customer.id | Supplier.id
    entityType   String    // CUSTOMER | SUPPLIER
    checkType   String    // KYC | AML | SANCTIONS | EXPORT_CONTROL | PEP
    status      String    // CLEAR | FLAGGED | PENDING | FAILED
    checkedAt   DateTime
    checkedBy   String    // User.id or 'system'
}

```

```

    provider      String?    // screening provider name
    score         Int?       // risk score 0-100
    flags         String[]   // specific flags raised
    notes         String?
    expiresAt     DateTime?
    version       Int        @default(1)
    createdAt     DateTime   @default(now())
}

model ExportControlRecord {
    id            String      @id @default(uuid())
    tenantId      String
    productId     String
    hsCode        String
    eccn          String?    // US Export Classification Control Number
    restrictions  String[]   // EMBARGOED_COUNTRIES list
    licenceRequired Boolean @default(false)
    notes        String?
    version       Int        @default(1)
    createdAt     DateTime   @default(now())
    updatedAt     DateTime   @updatedAt
}

enum ContractType {
    CUSTOMER_FRAMEWORK SUPPLIER EMPLOYMENT NDA SERVICE REGULATORY
}

enum ContractStatus {
    DRAFT UNDER_REVIEW APPROVED ACTIVE EXPIRED TERMINATED
}

```

---

## M08.3 — API CONTRACT

| None      |                          |                       |        |
|-----------|--------------------------|-----------------------|--------|
| METHOD    | PATH                     | PERMISSION            | STATUS |
| POST      | /api/legal/contracts     | legal.contract.create |        |
| SPECIFIED |                          |                       |        |
| GET       | /api/legal/contracts     | legal.contract.read   |        |
| SPECIFIED |                          |                       |        |
| GET       | /api/legal/contracts/:id | legal.contract.read   |        |
| SPECIFIED |                          |                       |        |

```
POST    /api/legal/contracts/:id/approve    legal.contract.approve
SPECIFIED
POST    /api/legal/contracts/:id/sign       legal.contract.sign
SPECIFIED

GET      /api/legal/compliance/:entityId    legal.compliance.read
SPECIFIED
POST     /api/legal/compliance/screen       legal.compliance.screen
SPECIFIED

GET      /api/legal/export-control/:productId legal.export.read
SPECIFIED
POST     /api/legal/export-control         legal.export.manage
SPECIFIED
```

---

## M08.4 — GRADUATION ROADMAP

| None                          |         |        |                                 |
|-------------------------------|---------|--------|---------------------------------|
| FEATURE                       | CURRENT | TARGET | PRIORITY                        |
| Contract Prisma model         | S0      | S4     | Sprint 3 Legal                  |
| ComplianceRecord model        | S0      | S4     | Sprint 2 Legal (Governance dep) |
| KYC/AML screening integration | S0      | S4     | Sprint 3 Legal                  |
| Sanctions check API           | S0      | S4     | Sprint 2 Legal (Governance dep) |
| Export control records        | S0      | S3     | Sprint 4                        |
| Contract signing workflow     | S0      | S3     | Sprint 4                        |
| Contract expiry alerts        | S0      | S3     | Sprint 4                        |
| Legal UI /os/legal            | S0      | S3     | Sprint 4                        |

---

## MODULE 09 — HR + PAYROLL

**Band:** S0 — not built as domain product  
**Owner Domain:** People  
**Status:** No HR domain implementation found

---

## M09.1 — POSITION IN ORGANISM

HR owns the people system of record. Every employee is a system actor — their identity flows from HR into Identity (M16). Payroll generates journals in Finance (M01) and payment instructions in HPay (M03). Leave management affects production capacity and Procurement staffing.

---

## M09.2 — ENTITY MODEL

```
None
model Employee {
  id                String                @id @default(uuid())
  tenantId          String
  employeeNumber    String                // system-generated
  userId            String                @unique // Identity M16 User.id
  firstName         String
  lastName          String
  email             String
  phone             String?
  jobTitle          String
  department        String
  managerId         String?              // Employee.id
  employmentType    EmploymentType      // FULL_TIME | PART_TIME | CONTRACT | INTERN
  status            EmployeeStatus      // ACTIVE | ON_LEAVE | TERMINATED | SUSPENDED
  startDate         DateTime
  endDate           DateTime?
  salary            BigInt                // annual gross, minor units
  currencyCode      String
  bankAccountId     String?              // HPay Beneficiary.id
  countryCode       String
  taxIdentifier      String?
  version           Int                  @default(1)
  createdAt         DateTime              @default(now())
  updatedAt         DateTime              @updatedAt
  createdBy         String
  deletedAt         DateTime?

  manager           Employee?            @relation("EmployeeHierarchy", fields:
[managerId], references: [id])
  reports           Employee[]           @relation("EmployeeHierarchy")
  leaveRequests     LeaveRequest[]
  payrollEntries    PayrollLine[]
}
```

```

model LeaveRequest {
    id            String          @id @default(uuid())
    tenantId      String
    employeeId    String
    type          LeaveType       // ANNUAL | SICK | PARENTAL | UNPAID |
PUBLIC_HOLIDAY | COMPASSIONATE
    status        LeaveStatus     // PENDING | APPROVED | REJECTED | CANCELLED
    startDate     DateTime
    endDate       DateTime
    days          Decimal         @db.Decimal(5, 1)
    reason        String?
    approvedBy    String?
    approvedAt    DateTime?
    rejectedBy    String?
    rejectedReason String?
    version       Int             @default(1)
    createdAt     DateTime        @default(now())
    updatedAt     DateTime        @updatedAt

    employee      Employee        @relation(fields: [employeeId], references: [id])
}

model PayrollRun {
    id            String          @id @default(uuid())
    tenantId      String
    runNumber     String          // system-generated
    periodCode    String          // e.g. "2026-08"
    status        PayrollStatus   // DRAFT | CALCULATED | APPROVED | PAID | REVERSED
    runDate       DateTime
    paymentDate   DateTime
    totalGross    BigInt
    totalTax      BigInt
    totalNet      BigInt
    currencyCode  String
    approvedBy    String?
    approvedAt    DateTime?
    journalEntryId String?
    idempotencyKey String         @unique
    version       Int             @default(1)
    createdAt     DateTime        @default(now())
    updatedAt     DateTime        @updatedAt
    createdBy     String

    lines         PayrollLine[]
}

model PayrollLine {

```

```

    id          String      @id @default(uuid())
    payrollRunId String
    employeeId   String
    grossPay     BigInt
    taxAmount    BigInt
    deductions   BigInt      @default(0) // pension, insurance, etc.
    netPay       BigInt
    currencyCode String
    hpayStatus   String?     // PENDING | SUBMITTED | PAID | FAILED

    payrollRun PayrollRun @relation(fields: [payrollRunId], references: [id])
    employee   Employee   @relation(fields: [employeeId], references: [id])
}

enum EmploymentType {
    FULL_TIME PART_TIME CONTRACT INTERN
}

enum EmployeeStatus {
    ACTIVE ON_LEAVE TERMINATED SUSPENDED
}

enum LeaveType {
    ANNUAL SICK PARENTAL UNPAID PUBLIC_HOLIDAY COMPASSIONATE
}

enum LeaveStatus {
    PENDING APPROVED REJECTED CANCELLED
}

enum PayrollStatus {
    DRAFT CALCULATED APPROVED PAID REVERSED
}

```

---

## M09.3 — KEY WORKFLOW: PAYROLL RUN

**Trigger:** Monthly scheduled job or `POST /api/hr/payroll/runs`

**Idempotency key:** `periodCode` + `tenantId` — one run per period per tenant

None

1. Auth: HR system account or HRManager
2. Check: no existing PayrollRun for same periodCode (idempotency)

```

3. Fetch all ACTIVE employees for tenant
4. For each employee:
  a. Calculate grossPay: salary / 12 (monthly) + allowances - deductions
  b. Calculate taxAmount: call Tax Engine (employment income tax rate for
jurisdiction)
  c. Calculate netPay = grossPay - taxAmount - other deductions
5. Governance: CFO approval required for payroll run approval
6. BEGIN TRANSACTION:
  a. INSERT PayrollRun (status = DRAFT)
  b. INSERT PayrollLine per employee
  c. INSERT AuditLog
7. COMMIT
8. Submit for approval → on approval:
  a. INSERT JournalEntry:
    DR Salary Expense (6000)    totalGross
    DR Employer Tax (6100)     employerTax
    CR Salary Payable (2100)    totalNet
    CR PAYE Payable (2200)     taxAmount
  b. For each employee: POST /api/hpay/payments (netPay to employee bank account)
  c. UPDATE PayrollRun.status = PAID
  d. Emit: payroll.paid { payrollRunId, periodCode, totalNet, employeeCount }

```

## M09.4 — API CONTRACT

| None   |                                  |                       |           |
|--------|----------------------------------|-----------------------|-----------|
| METHOD | PATH                             | PERMISSION            | STATUS    |
| POST   | /api/hr/employees                | hr.employee.create    | SPECIFIED |
| GET    | /api/hr/employees                | hr.employee.read      | SPECIFIED |
| GET    | /api/hr/employees/:id            | hr.employee.read      | SPECIFIED |
| PATCH  | /api/hr/employees/:id            | hr.employee.update    | SPECIFIED |
| DELETE | /api/hr/employees/:id            | hr.employee.terminate | SPECIFIED |
| POST   | /api/hr/leave                    | hr.leave.request      | SPECIFIED |
| GET    | /api/hr/leave                    | hr.leave.read         | SPECIFIED |
| POST   | /api/hr/leave/:id/approve        | hr.leave.approve      | SPECIFIED |
| POST   | /api/hr/leave/:id/reject         | hr.leave.approve      | SPECIFIED |
| POST   | /api/hr/payroll/runs             | hr.payroll.run        | SPECIFIED |
| GET    | /api/hr/payroll/runs             | hr.payroll.read       | SPECIFIED |
| GET    | /api/hr/payroll/runs/:id         | hr.payroll.read       | SPECIFIED |
| POST   | /api/hr/payroll/runs/:id/approve | hr.payroll.approve    | SPECIFIED |

# M09.5 — GRADUATION ROADMAP

| None                       |         |        |                           |
|----------------------------|---------|--------|---------------------------|
| FEATURE                    | CURRENT | TARGET | PRIORITY                  |
| Employee Prisma model      | S0      | S4     | Sprint 1 HR               |
| Leave management           | S0      | S3     | Sprint 2 HR               |
| Payroll run (monthly)      | S0      | S4     | Sprint 2 HR (Finance dep) |
| Payroll → HPay integration | S0      | S4     | Sprint 3 HR               |
| HR UI /os/hr               | S0      | S3     | Sprint 2 HR               |
| Employee onboarding flow   | S0      | S3     | Sprint 3 HR               |
| Org chart                  | S0      | S2     | Sprint 4 HR               |

# MASTER SYSTEM ROADMAP

## PRIORITY SEQUENCE — BUILD ORDER

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| None                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| PHASE 0 – Foundation Integrity (IMMEDIATE – Days 1-14)                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <div>[ ] Fix /api/v2/* auth scope – actorId/actorRole = null is a live security gap</div> <div>[ ] Remove mock KPIs from all UI pages (847 accounts, \$4.82M pipeline, etc.)</div> <div>[ ] Unify dual Lead table – single canonical CRM Lead Prisma model</div> <div>[ ] Update module-registry.json honest status flags</div> <div>[ ] Verify double-entry enforcement in Finance Journal model</div> <div>[ ] Confirm Chart of Accounts and FiscalPeriod models exist and are correct</div> |
| PHASE 1 – CRM as Single Commercial System of Record (Days 15-45)                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <div>[ ] Lead qualify + convert workflows (with atomic transaction + governance)</div> <div>[ ] Contact entity CRUD under Customer</div> <div>[ ] CreditLimit Prisma model + API</div> <div>[ ] CrmActivity all types implemented</div> <div>[ ] Customer 360 page: real Prisma data only</div> <div>[ ] HarvyX import job – idempotent, externalId-keyed, one-way</div>                                                                                                                       |
| PHASE 2 – Quote-to-Cash Thin Slice (Days 46-90)                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <div>[ ] Tax Engine (M04) – TaxJurisdiction + TaxCode + calculate API</div> <div>[ ] PriceList + PriceListEntry Prisma models</div>                                                                                                                                                                                                                                                                                                                                                            |

- [ ] Quote + QuoteLine with Tax Engine integration (block if tax unavailable)
- [ ] Quote approval workflow + governance on budget authority
- [ ] SalesOrder + SalesOrderLine from approved Quote (idempotency key mandatory)
- [ ] SalesOrder → Finance AR handoff (invoice.requested domain event)
- [ ] AR Invoice creation from CRM event (Finance M01)
- [ ] Neural Governance wired on Quote + Order execution paths (verified)
- [ ] Legal/Compliance: ComplianceRecord model + sanctions check for order creation

#### PHASE 3 – Source-to-Pay Thin Slice (Days 91-150)

---

- [ ] Supplier master (M07) – Prisma model + CRUD + status
- [ ] Purchase Requisition lifecycle + approval
- [ ] Purchase Order with governance (all five checks)
- [ ] Goods Receipt Note
- [ ] Three-way match logic (PO → GRN → AP Bill)
- [ ] AP Bill in Finance (M01) – from GRN event
- [ ] HPay (M03) – PaymentInstruction model + outbound payment workflow
- [ ] AP Bill approval → HPay payment initiation
- [ ] HPay → Finance GL journal (payment.completed event → Journal post)

#### PHASE 4 – Operations + Controls (Q2)

---

- [ ] Returns (RMA) workflow in CRM
- [ ] Support Tickets with SLA monitoring
- [ ] Full RBAC matrix implemented per-route (all modules)
- [ ] Rate limiting on AI-touching endpoints
- [ ] Governance audit log verified for ALL module mutations
- [ ] Period Close workflow (Finance M01)
- [ ] Financial reports: Trial Balance, P&L, AR Aging
- [ ] HR: Employee model + Leave management
- [ ] Contract management (M08) – basic lifecycle
- [ ] Gate 5 checklist: load test, DR plan, security pen test

#### PHASE 5 – Intelligence + Scale (H2)

---

- [ ] Payroll run + HPay integration (M09)
- [ ] CPQ AI (price.py) connected to live Prisma data
- [ ] Data Ocean Bronze: CRM + Finance journal events → Kafka
- [ ] Data Ocean Silver: cleaned dimensional models
- [ ] Data Ocean Gold: aggregated KPIs for HARVEY engine
- [ ] Board pack KPIs from real Finance + CRM data
- [ ] Multi-currency FX revaluation (Finance M01)
- [ ] Marketing campaign attribution (M06) from real CRM conversion data
- [ ] Subscription / Loyalty / Rebates (CRM M05) – after commercial path stable
- [ ] Trade Promotion Management domain (Rebates separation from CRM)

---

## MODULE STATUS DASHBOARD (honest, August 2026)

| None   |                    |         |        |                         |
|--------|--------------------|---------|--------|-------------------------|
| MODULE | NAME               | CURRENT | TARGET | PHASE                   |
| M01    | Finance Core       | S1/UNKN | S4     | Phase 1-3               |
| M03    | HPay               | S1/UNKN | S4     | Phase 3                 |
| M04    | Tax Engine         | S0      | S4     | Phase 2 (BLOCKER)       |
| M05    | CRM + Sales        | S3→S4   | S4+G5  | Phase 0-2               |
| M06    | Marketing + HarvyX | S1      | S3     | Phase 1                 |
| M07    | Procurement        | S0      | S4     | Phase 3                 |
| M08    | Legal + Compliance | S0      | S4     | Phase 2-4               |
| M09    | HR + Payroll       | S0      | S4     | Phase 4                 |
| M10    | Inventory + WMS    | S0      | S4     | Phase 3+                |
| M13    | Logistics + Fleet  | S0      | S3     | Phase 4+                |
| M16    | Identity + Auth    | S3      | S4     | Phase 0 (fix /api/v2)   |
| M19    | Audit Log          | S2      | S4     | Phase 1                 |
| M20    | Notifications      | S2      | S4     | Phase 1                 |
| M22    | Neural Governance  | S2      | S4     | Phase 2 (verify wiring) |

## SYSTEM-WIDE PROHIBITIONS (CURSOR MUST NOT)

|      |                                                                                                  |
|------|--------------------------------------------------------------------------------------------------|
| None |                                                                                                  |
| X    | Trust tenantId from request body – always from JWT server-side                                   |
| X    | Post unbalanced journal entries (Finance M01)                                                    |
| X    | Estimate tax when Tax Engine unavailable – block the operation                                   |
| X    | Duplicate financially material documents (invoice, PO, payment) – idempotency key mandatory      |
| X    | Use Decimal for monetary values where BigInt minor units is the standard                         |
| X    | Mark any feature S4 without full auth + audit + governance on execution path verified            |
| X    | Hard-delete Customer, Lead, Deal, Quote, Order, Invoice, Journal, Employee – soft delete only    |
| X    | Write actorId = null or actorRole = null to any audit log                                        |
| X    | Fabricate AI results when AI service unavailable – null is correct                               |
| X    | Store card numbers, CVVs, bank credentials in business module tables – HPay token reference only |
| X    | Present scaffold routes as production features                                                   |
| X    | Show mock KPI numbers on any UI page                                                             |
| X    | Mark Gate 5 without load test, DR test, and security pen test                                    |

# DOCUMENT MAINTENANCE

When any module implementation changes:

1. Update Section 3 (Honest State) of that module — adjust status labels
2. Update MODULE STATUS DASHBOARD above
3. Update `module-registry.json` canonical flags
4. Update `MASTER.md` if any architectural principle changes

Do not update this file to reflect aspirational state. Update it to reflect actual state.

**Last updated:** August 2026

**Authority:** Harvics OS build constitution — compiled from all source materials

**Next review:** On completion of Phase 0 (Days 1-14)

---

**END OF HARVICS\_OS\_CURSOR\_MASTER.md**